

Guía de estudio ilustrada

Administración de memoria: paginación y segmentación

Basada en el documento de la Unidad 1 – Sistemas Operativos 2

Idea clave (subrayar)

Todo el documento gira alrededor de **tres funciones** que el sistema operativo debe cumplir sobre la memoria:

1. **Relocalización:** mover un programa de lugar en memoria sin que deje de funcionar.
2. **Protección:** que un proceso no pueda leer/escribir memoria de otro.
3. **Compartición:** que dos procesos puedan usar el mismo bloque cuando convenga.

Cada mecanismo del curso se evalúa contra estas tres funciones.

Índice

1. Evolución de los mecanismos de memoria	2
1.1. Carga estática	2
1.2. Carga dinámica: registros de relocalización/límite	2
2. Direcciones y tamaño de palabra	3
3. Paginación	3
3.1. Los tres componentes del mecanismo	3
3.2. Dos formas de calcular la dirección física	4
3.3. Ejemplo completo (Figura 3 del documento)	5
4. Enlace estático y dinámico	5
5. Segmentación	6
6. Sistemas híbridos (segmentación + paginación)	7
7. Paginación multinivel	8

1. Evolución de los mecanismos de memoria

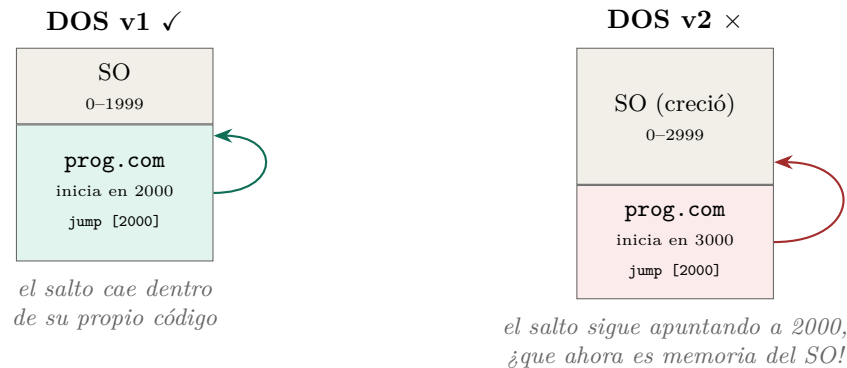
El hardware de administración de memoria evolucionó en tres fases: **carga estática** → **carga dinámica** (registros de relocación/límite) → **paginación**.

1.1. Carga estática

En los primeros sistemas (monousuario) no había hardware de protección. El programador debía conocer *de antemano* la dirección física donde se cargaría el programa, y escribía **referencias absolutas** tanto para datos (lecturas) como para código (saltos).

Ejemplo desarrollado: los .COM de DOS

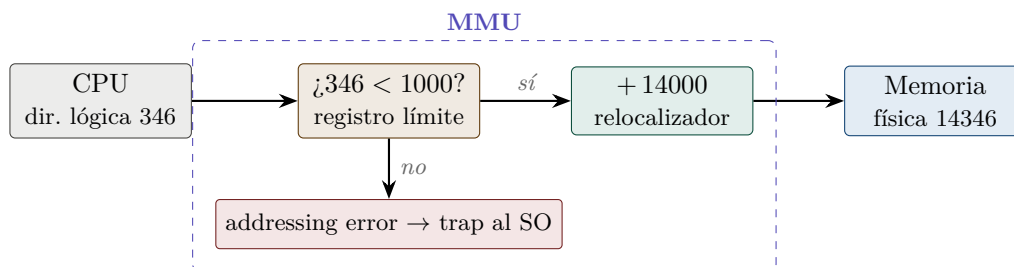
Los ejecutables .COM estaban programados para iniciar en una posición fija, que dependía de cuánta memoria ocupaba esa versión de DOS. Al cambiar de versión, el SO ocupaba distinta cantidad de memoria, la posición inicial disponible cambiaba y los programas de la versión anterior **dejaban de funcionar**: sus saltos absolutos caían en memoria equivocada.



1.2. Carga dinámica: registros de relocación/límite

La solución: el programa usa **direcciones relativas** (su memoria “empieza en 0”) y el MMU (Memory Management Unit) traduce en cada acceso usando dos registros:

- **Registro límite**: cuánta memoria tiene asignada el proceso. Si la dirección lógica $\geq$ límite $\Rightarrow$ *addressing error* (protección).
- **Registro relocador (base)**: dónde empieza realmente el proceso en memoria física. Se *suma* a la dirección lógica (relocalización).



Idea clave (subrayar)

El límite y el relocalizador **forman parte del contexto del proceso**: en cada cambio de contexto el SO debe restaurarlos con los valores del proceso que entra a ejecutar, igual que sus demás registros. Si el SO quiere mover el proceso, solo cambia el relocalizador; el programa ni se entera.

Ejemplo desarrollado: registros base en los primeros Intel

La relocalización se hacía con registros de segmento (DS datos, CS código, SS pila). Las instrucciones

```
mov ax, [FF]      jump [5]
```

equivalen implícitamente a

```
mov ax, DS:[FF]   jump CS:[5]
```

El programador solo piensa en direcciones relativas.

2. Direcciones y tamaño de palabra

- **Tamaño de palabra**: número de bits que se transfieren de memoria a un registro en una operación (el ancho del bus del sistema): 8, 16, 32, 64 bits.
- Una **dirección** puede referirse a un byte, a un grupo de bytes o a una palabra, según el diseño del sistema.

Esto importa porque en paginación la dirección se *parte en campos de bits*, y hay que saber cuántos bits tiene la dirección completa.

3. Paginación

Definición: paginación

Dividir la memoria en partes **iguales y de tamaño fijo**. Del lado lógico se llaman **páginas**; del lado físico, **marcos de página** (frames). La página es la *unidad mínima de asignación*: toda petición de memoria se redondea hacia arriba a páginas completas.

Definición: fragmentación interna

Desperdicio de memoria *dentro* de un bloque de tamaño N entregado a un proceso que pidió $K < N$: se pierden $N - K$ dentro del bloque, y nadie más puede usarlos.

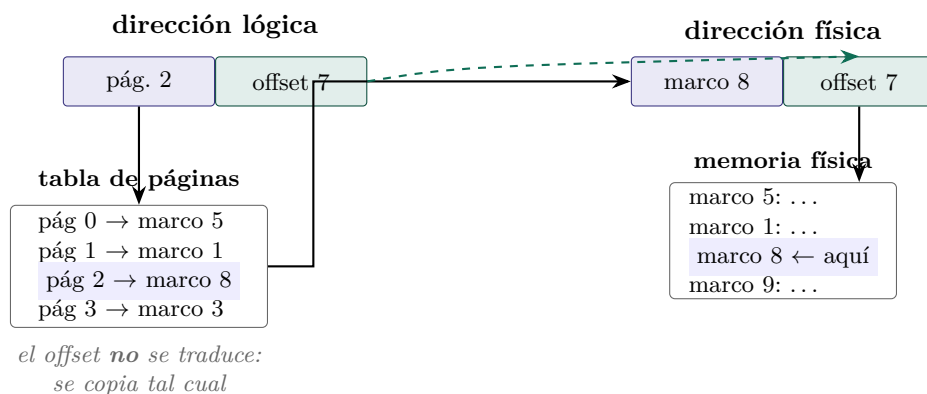
Ejemplo desarrollado: fragmentación interna

Páginas de 10 KB, el proceso pide 45 KB $\Rightarrow$ el SO entrega $\lceil 45/10 \rceil = 5$ páginas = 50 KB. En la última página quedan $50 - 45 = 5$ KB desperdiciados.

3.1. Los tres componentes del mecanismo

1. **Dirección lógica dividida en dos partes**: número de página (bits altos) + desplazamiento u *offset* (bits bajos).

2. **Tabla de páginas** (una por proceso): para cada página lógica indica en qué marco físico está, además de PID propietario y permisos.
3. **Page Table Pointer**: registro del MMU que apunta a la tabla de páginas del proceso *en ejecución*; se restaura en cada cambio de contexto.



3.2. Dos formas de calcular la dirección física

- 1) **Por suma.** La tabla guarda la *dirección física completa* del marco:

$$dirFísica = frameDir + offset$$

Desventajas: entradas más grandes en la tabla y una suma por cada acceso.

- 2) **Por yuxtaposición.** La tabla guarda solo el *número* de marco:

$$dirFísica = dirBase + frameNumber \cdot pageSize + offset$$

Como $pageSize = 2^{bits}$, la multiplicación se sustituye por un corrimiento de bits:

$$frameNumber \ll bits$$

que es muchísimo más eficiente. Además, usualmente los marcos inician en la dirección física 0 (el kernel vive en memoria alta), así que $dirBase = 0$.

Ejemplo desarrollado: yuxtaposición con números

Páginas de 4 KB $\Rightarrow$ offset de 12 bits. Marco = 8 = 1000_2 , offset = 7 = 000000000111_2 .

$$\underbrace{1000}_{\text{marco}} \underbrace{000000000111}_{\text{offset (12 bits)}} = 1000000000000111_2 = 32775$$

Comprobación: $8 \times 4096 + 7 = 32768 + 7 = 32775$. ✓ No hubo suma real: solo se *concatenaron* los bits.

Cuidado / típica pregunta de examen

Balance del tamaño de página (configurable al iniciar el SO):

- Páginas **grandes** $\rightarrow$ tabla pequeña ✓, pero más fragmentación interna ×.
- Páginas **pequeñas** $\rightarrow$ poco desperdicio ✓, pero tablas enormes y búsquedas más costosas ×.

3.3. Ejemplo completo (Figura 3 del documento)

Sistema teórico: direcciones de **4 bits** (máximo $2^4 = 16$ direcciones), palabra de 8 bits. Se usan **2 bits para número de página** y **2 bits para offset** $\Rightarrow$ 4 páginas de 4 direcciones cada una. El proceso X guarda el string “*este es un texto*” (16 caracteres, uno por dirección).

Dirección lógica		Página	Contenido
binario	decimal		
0000–0011	0–3	00	e s t e
0100–0111	4–7	01	e s
1000–1011	8–11	10	u n t
1100–1111	12–15	11	e x t o

La tabla de páginas del proceso X guarda, por cada página: **PID propietario**, **permisos** (accesos) y **dirección base** del marco:

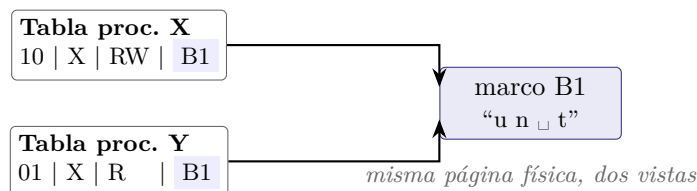
No.	PID	Acceso	Base (marco)
00	X	RW	EF
01	X	RW	00
10	X	RW	B1
11	X	RW	FF

Idea clave (subrayar)

Para el proceso, las direcciones son un rango continuo (0000 a 1111); en memoria física las páginas pueden estar **en cualquier orden y lugar**. Si el SO mueve una página física, solo actualiza *una entrada* de la tabla: el proceso sigue viendo las mismas direcciones lógicas. Eso es la **relocalización** en paginación.

Compartición entre procesos. Si un proceso Y necesita usar la página 10 del proceso X, X la comparte explícitamente y en la tabla de Y se agrega una entrada apuntando al *mismo marco* (B1). Dos observaciones del documento (típicas de examen):

1. La **dirección lógica no es la misma** en ambos procesos: cada tabla asigna la entrada según su propio estado. X la ve como su página 10; Y puede verla como su página 01.
2. La entrada en la tabla de Y indica que el **propietario es X** y que Y tiene **solo lectura** (R). Eso es la **protección** aplicada a la **compartición**.



4. Enlace estático y dinámico

Caso real donde brilla la compartición por paginación: las **librerías**.

- **Enlace estático:** el ejecutable contiene *dentro de sí* todas las librerías. Distribución simple (un solo archivo), pero si dos programas usan la librería A, ésta queda cargada **dos veces** en memoria: desperdicio de memoria y de tiempo de carga.

- **Enlace dinámico:** el ejecutable contiene solo su código propio; las librerías viven en archivos aparte (A.dll, B.dll, C.dll) que se cargan *una sola vez* y se comparten entre programas.



Secuencia de carga (ejemplo del documento). Al ejecutar X: pide la librería A → no está en memoria → el SO la carga y le comparte su dirección; luego lo mismo con C. Al ejecutar Y: pide A → **ya está cargada** → el SO solo le comparte la dirección; pide B → se carga.

Costo del enlace dinámico. Mantenimiento más complejo: las nuevas versiones de una librería deben mantener **compatibilidad hacia atrás** (en interfaz y funcionamiento) para no romper a los programas que la usan.

Código típico (lo que el compilador genera por nosotros). El programador escribe algo como:

```
int a1(int a, X c); extern "a.dll"; ... int z = a1(0, null);
```

y el compilador lo transforma en llamadas explícitas al SO:

```
handle = loadLibrary("a.dll"); a1 = getProc(handle,"a1"); z = a1(0,null); unloadLibrary(handle);
```

Definición: conteo de referencias

`unloadLibrary` **no** elimina la librería de memoria: el SO lleva un *contador de procesos* que la usan. X carga A ⇒ contador = 1; Y la pide ⇒ contador = 2; X la descarga ⇒ contador = 1 (sigue en memoria); Y la descarga ⇒ contador = 0 ⇒ ahora sí puede eliminarse sin afectar a nadie.

En las tablas de páginas (Figura 7 del documento), las páginas de librerías aparecen con **propietario = sistema (sys)** y permisos **RX** (leer y ejecutar, nunca escribir) para cada proceso que las comparte; las páginas propias de cada proceso siguen siendo RW.

5. Segmentación

La paginación cumple las tres funciones, pero sufre **fragmentación interna** y además el programador piensa la memoria como *bloques semánticos* (código, datos, pila...), no como páginas uniformes.

Definición: segmentación

Mismo principio que la paginación, pero con bloques **semánticamente relacionados y de tamaño variable**: cada segmento mide exactamente lo que el proceso pidió. Eso **elimina la fragmentación interna**.

Componentes (análogos a la paginación): dirección lógica = (número de segmento, offset); **tabla de segmentos** con dirección base y *tamaño* de cada segmento; **Segment Table Pointer**.

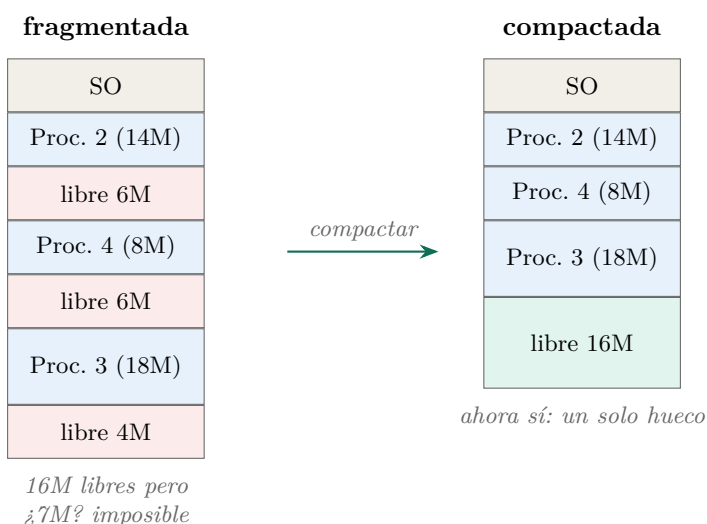
Traducción: (1) buscar el segmento en la tabla; (2) verificar que $offset < tamaño$ del segmento (aquí vive la protección: el límite ahora es *por segmento*); (3) $dirFísica = base + offset$ (aquí siempre hay suma: los segmentos son variables, no se puede yuxtaponer).

Definición: fragmentación externa

Pequeños bloques *libres* intercalados entre bloques ocupados: aunque la suma total libre sea N , el sistema no puede atender solicitudes de tamaño N (ni menores que N pero mayores que el hueco más grande), porque la memoria libre está **dispersa**.

Ejemplo desarrollado: fragmentación externa (Figura 9)

Procesos 2, 3 y 4 ocupan 14, 18 y 18 MB. Quedan libres tres huecos: 6, 6 y 4 MB = **16 MB libres en total**. Llega una solicitud de **7 MB** y... no se puede atender: ningún hueco individual alcanza.



Cuidado / típica pregunta de examen

La **compactación** resuelve la fragmentación externa moviendo todos los bloques ocupados a un extremo... pero mientras se ejecuta, **los procesos de usuario y del sistema deben detenerse** hasta terminar de relocalizar todo. Impacto directo al rendimiento. (Comparar: paginación → fragmentación interna; segmentación → fragmentación externa.)

6. Sistemas híbridos (segmentación + paginación)

La segmentación, además de la fragmentación externa, complica el control por la *variabilidad de tamaños*. Los sistemas híbridos toman lo mejor de ambos: **primer nivel de segmentos, segundo nivel de paginación**. El programador pide segmentos; transparentemente cada segmento se divide en páginas.

La dirección lógica ahora tiene **tres partes**:



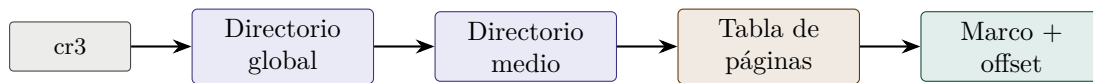
Traducción: (1) el Seg # se busca en la tabla de segmentos, que ya **no** contiene la dirección del segmento sino la dirección de *la tabla de páginas de ese segmento*; (2) el Page # se busca en esa tabla de páginas → dirección base de la página; (3) se suma el offset → dirección física.

Idea clave (subrayar)

Si hay fragmentación interna, queda limitada a **la última página de cada segmento**. Y no hay fragmentación externa porque físicamente todo se asigna en páginas iguales.

7. Paginación multinivel

El concepto de dos niveles se generaliza a una **jerarquía de tablas de páginas**: la dirección virtual se parte en varios índices (p. ej. *Global Directory* → *Middle Directory* → *Page Table* → *Offset*), y un registro (como `cr3` en x86) apunta a la tabla raíz del proceso.



cada campo de la dirección virtual indexa un nivel

Idea clave (subrayar)

La ventaja: las tablas de niveles inferiores se crean **dinámicamente, solo cuando se necesitan**. Si un proceso usa apenas una fracción de su espacio de direcciones (el caso normal), no se gasta memoria en tablas para las regiones que nunca toca.

Resumen comparativo

	Reg. base/límite	Paginación	Segmentación
Unidad de asignación	bloque continuo	página (fija)	segmento (variable)
Relocalización	cambiar registro base	actualizar tabla de páginas	actualizar tabla de segmentos
Protección	registro límite	PID + permisos por página	tamaño por segmento
Compartición	no práctica	entradas al mismo marco	entradas al mismo segmento
Problema principal	todo el proceso continuo	fragmentación interna	fragmentación externa
Solución evolutiva	paginación	páginas pequeñas / multinivel	híbrido seg.+pag. / compactación